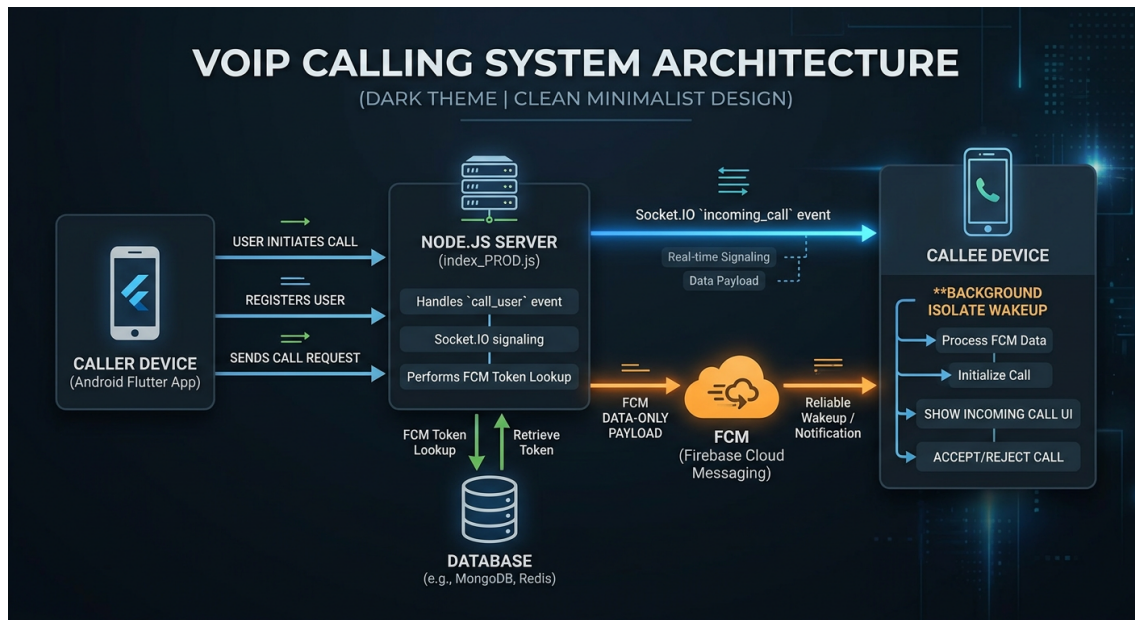


Mécanisme d'Appels Temps Réel Alanya / Alanya Work

Architecture FCM Data-Only • Telecom Natif Android • CallKit Fallback • WebRTC • Foreground Service •
Déduplication



Version	1.0 – 01/08/2026 (Africa/Douala)
Stack	Flutter 3.44.6 / Firebase Messaging / flutter_webrtc / Android ConnectionService / Node.js
Auteur	Dominique-BRIA / Equipe Alanya – Branche arena/019fbc55-alanya
Backend réf	github.com/Dominique-BRIA/backend-alanya@backup/pre-harmonisation-2026-07-24
Objectif	Comprendre le flux complet d'un appel de bout en bout, du clic Appeler à la bulle WhatsApp-like

Table des matières

1. Vue d'ensemble et enjeux
2. Backend – Préparation de l'appel (POST /api/calls)
3. Double canal : Socket.IO + FCM Data-Only
4. Réveil en arrière-plan – Isolate Dart
5. Chemin Android principal : Telecom Natif
6. Fallback CallKit
7. Acceptation / Refus / Occupé
8. WebRTC – Négociation SDP et ICE
9. Foreground Service et persistance
10. Déduplication et timers (90s vs 2min)
11. Statuts précis frontend – Mapping UX
12. UI : bulles d'appel WhatsApp-like
13. Points de vigilance & améliorations
14. Annexes : extraits de code

1. Vue d'ensemble et enjeux

Alanya est une messagerie de type WhatsApp. Le module d'appel doit fonctionner même si l'application est tuée, en arrière-plan, écran verrouillé ou retirée de la mémoire. C'est le point le plus complexe d'une app VoIP : réveiller l'app puis afficher une UI système native avant même que Flutter ne soit visible.

Le mécanisme retenu repose sur 4 piliers :

- **FCM data-only** : un message Firebase sans notification visuelle, qui ne passe pas par le système, mais réveille directement l'isolate Dart en background.
- **Telecom natif Android** : ConnectionService déclaré dans AndroidManifest.xml, permet d'afficher l'écran d'appel système même par-dessus l'écran verrouillé.
- **CallKit / flutter_callkit_incoming** : fallback quand Telecom échoue ou non disponible.
- **WebRTC mesh** : pour audio/vidéo, gestion de groupe via `webrtc_group_mesh.dart` et session 1-1 via `webrtc_peer_session.dart`.

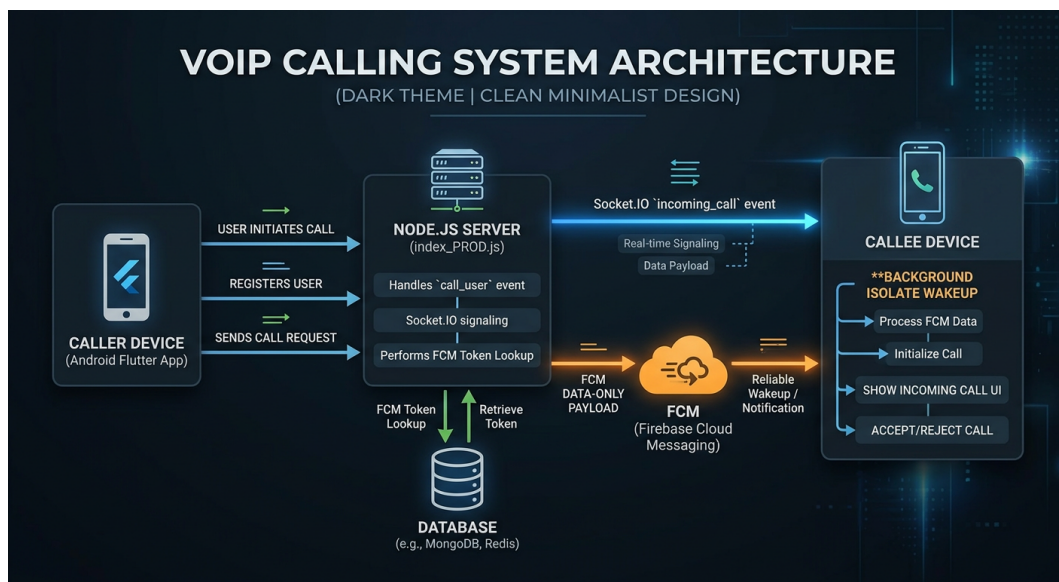


Figure 1 – Architecture globale double canal

2. Backend – Préparation de l'appel (POST /api/calls)

Le flow commence chez l'appelant A. Dans chat_screen.dart, tap sur la bulle d'appel → bottom sheet Audio/Vidéo → _startCall().

```
// Frontend - lib/features/chat/screens/chat_screen.dart
Future<void> _startCall(CallType type) async {
  final repo = CallsRepository();
  final callId = await repo.start(convId, type); // POST /api/calls
  callController.startOutgoing(callId, convId, type);
  Navigator.push(ActiveCallScreen(incoming:false));
}
```

Côté serveur (index_PROD.js:2013 – événement call_user) :

```
// Backend - extrait simplifié
POST /api/calls { convId, type: 'AUDIO'|'VIDEO' }
- assertParticipant(convId, userId)
- Vérif stale RINGING >2min => update ENDED
- Vérif BUSY: callParticipant où joinedAt!=null && leftAt==null && status in
[RINGING, ONGOING]
if busy return 409 BUSY
- Génère callId = uuid()
- Prisma: create call { id, convId, type, status:RINGING, initiator:userId, startedAt:
now() }
- call_history upsert + in_call = true pour participants
- Récup token FCM de B via device_registry
- Double émission parallèle :
io.to(socketIdB).emit('incoming_call', { callId, convId, callerId, callerName, type, ...
})
+ FCM data-only si token valide : { type:'incoming_call', callId, callerName, avatar,
callType }

GET /api/calls - 50 derniers (doit passer à 20 triés desc)
```

Point clé : le serveur ne décide pas encore du statut final MISSED/REJECTED. Il laisse RINGING puis clean via cron 2 minutes.

Bug backend actuel : les appels stale RINGING >2min sont marqués ENDED au lieu de MISSED/NO_ANSWER. Il faut distinguer : si jamais answeredAt==null et aucun reject → MISSED côté B, NO_ANSWER côté A.

3. Double canal : Socket.IO + FCM Data-Only

Pourquoi deux canaux ? Socket.IO est instantané si B est connecté, mais si l'app est tuée, le socket est mort. FCM data-only est la seule façon fiable de réveiller un appareil Android/iOS en Doze.

```
// lib/core/push_service.dart
FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);

Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage msg) async {
  await Firebase.initializeApp();
  final data = msg.data; // {type: incoming_call, callId, convId, ...}
  if (data['type']=='incoming_call'){
    await AlanyaTelecom.reportIncomingCall(data); // tente Telecom
    await showCallkitIncoming(data); // fallback
  }
}
```

Payload FCM must be **data-only**, pas de champ notification, sinon Android affiche une notif générique et ne lance pas l'isolate en priorité haute. Avec 'priority: high' + data-only, le système réveille même en battery saver (selon OEM).

4. Réveil en arrière-plan – Isolate Dart

FirebaseMessaging.onBackgroundMessage s'exécute dans un isolate séparé, sans UI. C'est main.dart:32. Il n'a pas accès au navigator. Il doit donc : 1) init Firebase, 2) init sonnerie, 3) essayer Telecom natif via MethodChannel, 4) fallback CallKit.

```
// main.dart
@pragma('vm:entry-point')
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage m) async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  final svc = NotificationService.instance;
  await svc.ensureInitialized();
  await svc.handleBackgroundIncomingCall(m.data);
}
```

Attention : cet isolate n'a pas le Provider CallController. Il utilise un CallRegistry singleton écrit en Kotlin + SharedPreferences pour partager callId.

5. Chemin Android principal : Telecom Natif

Android Telecom Native Path vs CallKit Fallback Flowchart

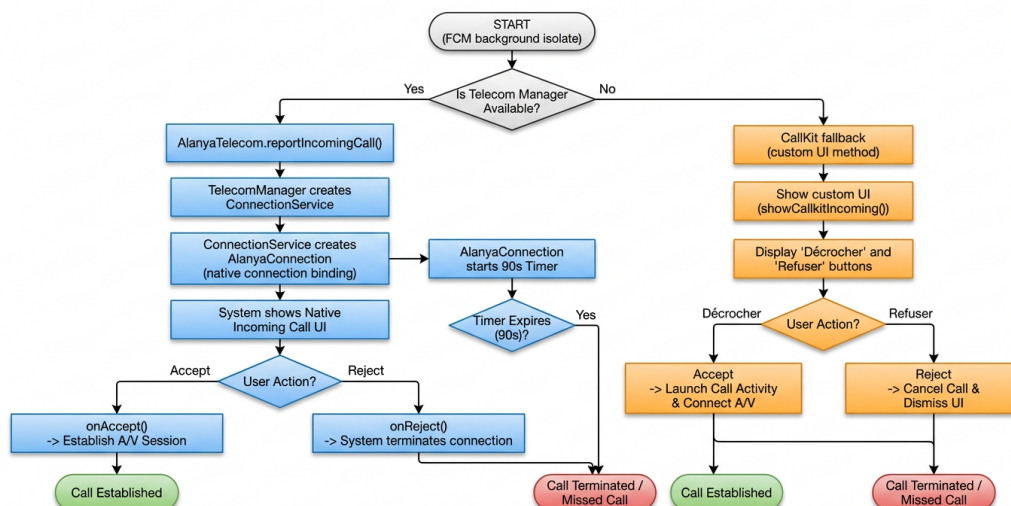


Figure 2 – Flux Telecom Natif vs CallKit fallback

Telecom est le mécanisme officiel pour les appels VoIP sur Android depuis API 23. Il faut déclarer un `ConnectionService` dans `AndroidManifest.xml`.

```
// AndroidManifest.xml
<service android:name=".telecom.AlanyaConnectionService"
android:permission="android.permission.BIND_TELECOM_CONNECTION_SERVICE">
<intent-filter><action android:name="android.telecom.ConnectionService"/></intent-filter>
</service>

// AlanyaConnectionService.kt:25
class AlanyaConnection: Connection() {
override fun onAnswer(){ // user décroche depuis l'UI système
// envoie broadcast -> Flutter CallController.acceptIncoming()
}
override fun onReject(){
// POST /decline-call + call_declined via socket
}
}

// Report appel
fun reportIncomingCall(data: Map) {
val telecomManager = getSystemService(TelecomManager)
val handle = ...
telecomManager.addNewIncomingCall(handle, extras) // affiche UI système plein écran
startTimer(90*1000) // auto-missed après 90s si pas de réponse -
AlanyaConnectionService.kt:40
}
```

Avantages Telecom : sonnerie système, plein écran sur lock screen, boutons natifs, intégration avec Bluetooth et volume d'appel. C'est ce que WhatsApp utilise.

6. Fallback CallKit

Si Telecom n'est pas disponible (permissions manquantes, OEM bloque, Android <23), on tombe sur flutter_callkit_incoming. C'est géré dans notification_service.dart:310.

```
// notification_service.dart:310
Future<void> showCallkitIncoming(Map data) async {
  await FlutterCallkitIncoming.showCallkitIncoming({
    'id': data['callId'],
    'nameCaller': data['callerName'],
    'avatar': data['callerAvatar'],
    'type': data['callType']=='VIDEO'?1:0,
    'duration': 90000, // même timer 90s max que Telecom
    'extra': data,
  });
  // Ecoute events
  FlutterCallkitIncoming.onEvent.listen((event){
    if(event.event==Event.actionCallAccept) handleAccept(event.body);
    if(event.event==Event.actionCallDecline) handleDecline(event.body); //
    notification_service.dart:354 & 480
  });
}
```

Sur refus depuis CallKit :

```
// notification_service.dart:480
sonnerie.stop()
try {
  socket.emit('call_declined', {callId})
  POST /decline-call {callId}
} catch
serveur: /api/decline-call index_PROD.js:1046 -> emit call_declined, pendingCalls cleanup,
declined, in_call=false
```

7. Acceptation / Refus / Occupé

Chemin acceptation (notification_service.dart:423) :

```
onAccept:
AudioCallScreen / CallScreen via navigatorKey.currentState
call_accepted emit - call_controller.dart:542
group_join si groupe
Serveur: index_PROD.js:2251 -> event accepted, status ONGOING, answeredAt=now()

Bug fixé dans ce cours (a11830e):
AVANT: ActiveCallScreen name = widget.incoming ? (cc.incoming?.displayTitle ?? 'Appel') :
activePeerName
-> Après accept, cc.incoming=null donc affiche 'Appel' chez B
APRES: if(widget.incoming && cc.incoming!=null) use displayTitle else use
participantNames[callerId] ?? activePeerName
+ call_controller.dart acceptIncoming() -> participantNames[callerId]=callerName,
joinedParticipantIds.add(callerId)
```

Logique BUSY : vérif côté serveur au POST si caller déjà en call (joinedAt != null && leftAt==null). Client : isBusy check via CallController.isBusy. Si B occupé, A doit recevoir statut BUSY (pas encore implémenté, actuellement ENDED).

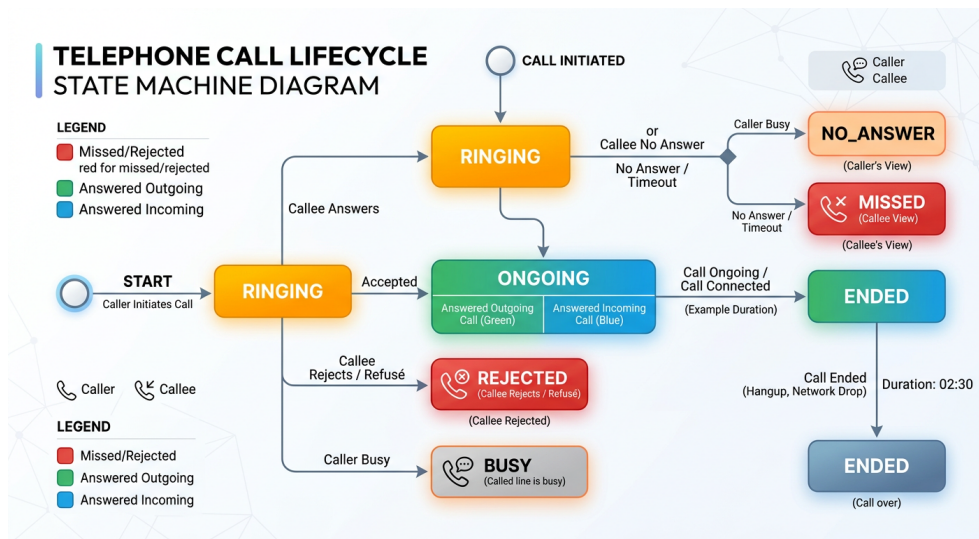
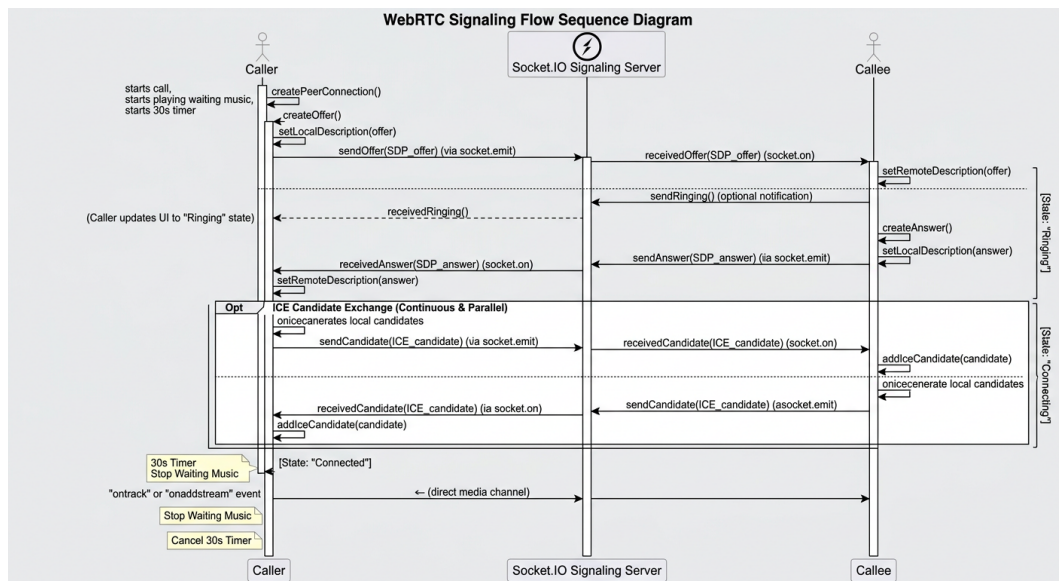


Figure 3 – Machine à états d'appel – statuts précis

8. WebRTC – Négociation SDP & ICE



Une fois l'appel accepté :

1. Musique d'attente stop, état ringing → connecting
2. Création `RTCPeerConnection` (`lib/features/calls/webrtc_peer_session.dart`)
3. Offer SDP créée par le callee qui a accepté (ou caller selon implé)
4. Échange via Socket.IO : events 'offer', 'answer', 'ice-candidate'
5. Premier flux distant `onTrack` → état connected + timer 30s guard
6. Datas pour mute, camera, speaker

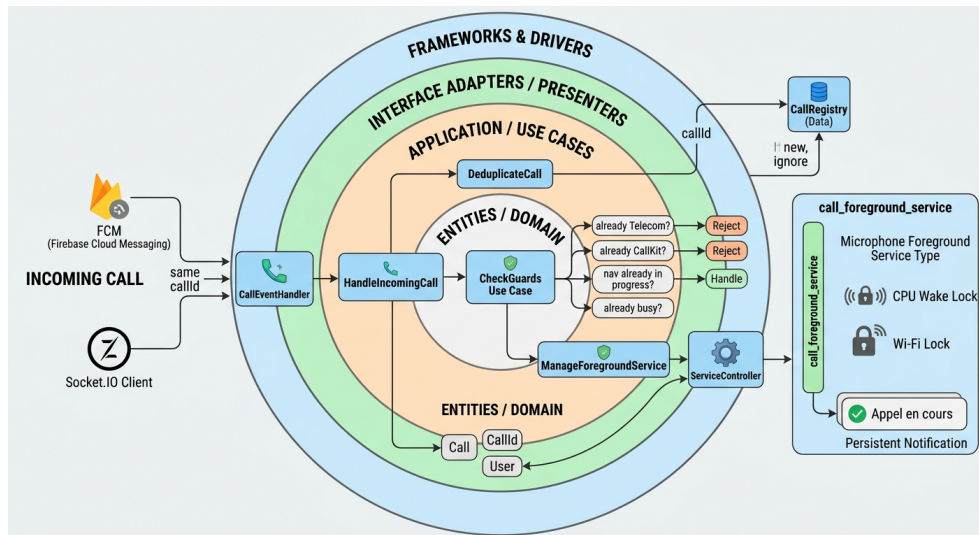
```

// webrtc_peer_session.dart (simplifié)
peerConnection = await createPeerConnection(config)
await peerConnection.addStream(localStream) // audio/video
offer = await peerConnection.createOffer()
await peerConnection.setLocalDescription(offer)
socket.emit('offer', {callId, sdp: offer.sdp})

socket.on('answer', (data) async {
  await peerConnection.setRemoteDescription(RTCSessionDescription(data.sdp, 'answer'))
})
socket.on('ice', (data) async {
  await peerConnection.addCandidate(RTCIceCandidate(...))
})
peerConnection.onTrack = (event) {
  remoteRenderer.srcObject = event.streams[0]
  callController.state = CONNECTED
}
  
```

Groupe : `webrtc_group_mesh.dart` crée N peer sessions (full mesh) – coûteux mais simple pour <6 participants. Chaque nouveau joine déclenche offer à tous.

9. Foreground Service & persistance



Sur Android, dès que l'app passe en background pendant un appel ONGOING, l'OS peut tuer le process. On lance donc un foreground service (call_foreground_service.dart:4) avec type microphone, wake lock CPU + Wi-Fi lock + notif persistante 'Appel en cours'.

```
// call_foreground_service.dart
Future<void> startCallForegroundService({required String callId, required String name})
async {
  FlutterForegroundTask.startService(
    notificationText: 'Appel en cours avec $name',
    notificationTitle: 'Alanya',
    callback: startCallback,
  )
  // WakeLock.enable()
  // WifiLock.acquire()
}

// arrêt à ENDED
FlutterForegroundTask.stopService()
```

10. Déduplication et timers

Problème : même callId arrive par deux canaux (Socket.IO + FCM). Il faut éviter double affichage.

Solution : CallRegistry singleton + guards dans socket_service.dart:122 :

- CallRegistry : Set callId déjà vus en mémoire + SharedPreferences
- check déjà Telecom affiché ? -> skip CallKit
- check déjà CallKit affiché ? -> skip Telecom
- check nav déjà en cours (activeCallId != null) -> si nouveau callId, mettre BUSY
- check déjà occupé (isGroupCall ou autre)

Timers :

Telecom timer 90s dans AlanyaConnectionService.kt:40 → auto MISSED si pas de réponse. Serveur cleanup 2min → transforme RINGING restant en ENDED (devrait être MISSED/NO_ANSWER). Frontend _loadCalls() garde timer local pour afficher 'Appel manqué' si endedAt==null && startedAt vieux >90s.

11. Statuts précis frontend – Mapping UX

Backend n'envoie que RINGING ONGOING ENDED MISSED REJECTED. Frontend bricole pour UX précise sans toucher backend (demande user).

```
// lib/features/calls/screens/calls_screen.dart + home_screen.dart + chat_screen.dart

enum PreciseCallStatus { MISSED, DECLINED, NO_ANSWER, BUSY, ANSWERED, REJECTED_OUT }

PreciseCallStatus _preciseCallStatus(CallRecord c){
  if(c.status==MISSED) return MISSED; // B n'a pas décroché -> chez B 'Appel manqué'
  if(c.status==REJECTED){
    if(c.isOutgoing) return REJECTED_OUT; // A a appelé, B a rejeté -> chez A 'Appel refusé'
    else return DECLINED; // chez B 'Appel rejeté'
  }
  if(c.status==ENDED){
    if(c.durationSec==null || c.durationSec==0){
      if(c.isOutgoing) return NO_ANSWER; // sortant non décroché
      else return MISSED;
    }
    return ANSWERED; // durée >0
  }
  // TODO BUSY: si duration==null && answeredAt==null && serveur busy flag
  return ANSWERED;
}

// Nuances définies:
- Appel manqué (Missed) : entrant sans réponse -> rouge + call_missed
- Appel rejeté (Declined/Rejected) : entrant refusé volontairement -> rouge + block
- Appel sans réponse (No Answer) : sortant non décroché -> rouge
- Occupé (Busy) : sortant tombe sur ligne occupée -> rouge
- Répondu : vert si sortant, bleu si entrant

Mapping couleur/icône:
Icon _callIconFor(status):
MISSED -> Icons.call_missed (rouge)
DECLINED -> Icons.block (rouge)
NO_ANSWER -> Icons.call_made (rouge)
BUSY -> Icons.call_missed (orange)
ANSWERED outgoing -> Icons.call_made (vert #22C55E)
ANSWERED incoming -> Icons.call_received (bleu #2196F3)
```

Exemple affiché dans liste d'appels (20 plus récents) :

```
_sortAndLimit(List<CallRecord> calls)
=> calls.sort((a,b)=>b.startedAt.compareTo(a.startedAt)).take(20)

Format date: dd/MM/yyyy HH:mm via intl
subtitle: "$_statusLabel · $_formatDateTime(startedAt) · ${duration ?? ''}"

Exemple:
Appel vocal entrant Rejeté · 15:12
Appel vocal sortant Sans réponse · 15:14
Appel vocal entrant Répondu · 15:14 · 0:13
```

12. UI : Bulles d'appel WhatsApp-like

L'utilisateur a demandé que les appels apparaissent non seulement dans l'onglet Appels mais aussi :

- Dans la liste conversations (`_lastCallPerConv`)
- Dans le fil de discussion comme sur WhatsApp (`_combined = [...messages, ...calls].sort(date)`)

```

// home_screen.dart
Map<String, CallRecord> _lastCallPerConv;
Future<void> _loadCalls() {
  final calls = await CallCache.instance.getAll() ?? await CallsRepository.history();
  for (c in _sortAndLimit(calls)) _lastCallPerConv[c.convId]=c; // garde plus récent par conv
}
// _tile affiche si call.startedAt > lastMessage.date:
Row(Icon(_callIconFor, color:_callColorFor, size:14), Text(_preciseLabel + ' . ' +
_formatShortTime))

// chat_screen.dart
List<dynamic> _combined; // messages + calls triés par date
DateTime _dateOfCombined(e) => e is Message ? e.timestamp : (e as CallRecord).startedAt
Widget _callBubbleInChat(CallRecord call){
  final isOutgoing = call.isOutgoing;
  return Align(
    alignment: isOutgoing? Alignment.centerRight : Alignment.centerLeft,
    child: InkWell(onTap: () => _showCallChoice(call), // bottom sheet Audio/Vidéo
    child: Container(
      constraints: BoxConstraints(maxWidth: 260),
      padding: EdgeInsets.symmetric(vertical:8, horizontal:12),
      margin: EdgeInsets.symmetric(vertical:2),
      decoration: BoxDecoration(
        color: _cardBg, // garde d'origine, pas sentBubbleColor
        borderRadius: BorderRadius.only(
          topLeft: isOutgoing?12:0, topRight: isOutgoing?0:12,
          bottomLeft:12, bottomRight:12), // forme WhatsApp queue haut
        ),
      child: Row(
        children:[
          Icon(_callIconFor(call), color:_callColorFor(call), size:14),
          SizedBox(width:8),
          Expanded(child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children:[
              Text(_callTitleInChat, style:TextStyle(fontSize:12, fontWeight:bold,
                color:_callColorFor)),
              Text(_callDetailInChat, style:TextStyle(fontSize:11, color:Colors.grey)),
            ]
          ))
        ]
      )
    )
  )
}

```

Couleurs : texte/icône rouge = manqué/rejeté, vert = sortant réussi, bleu = entrant réussi. Fond gardé _cardBg (consigne). Épaisseur réduite (margin 2, padding 8/12, icon 14). Bouton Appeler supprimé, tap → bottom sheet choix Audio/Vidéo qui lance appel.

13. Points de vigilance & améliorations

Points de vigilance actuels :

- Gradle 8.11.1 / AGP 8.9.1 / Kotlin 2.1.0 → warnings Flutter : support bientôt drop, passer à 8.14.0 / 8.11.1 / 2.2.20
- AppBar subtitle param n'existe pas → fix via title Column (corrigé 1b09b82)
- Backend take:50 vs frontend take:20 – aligner à 20 avec tri desc
- Stale RINGING → ENDED au lieu de MISSED/NO_ANSWER – implémenter timeout logique
- BUSY non géré côté serveur – créer statut BUSY et vérifier callee busy, pas seulement caller
- Droits RECORD_AUDIO, USE_SIP, POST_NOTIFICATIONS, FOREGROUND_SERVICE_MICROPHONE obligatoires
- OEM Xiaomi/Oppo killent foreground service – ajouter demande ignore battery optimization
- WebRTC mesh ne scale pas >6 participants – envisager SFU (Janus/LiveKit) plus tard

Améliorations futures :

- Backend : enum CallStatus +BUSY, +NO_ANSWER ; POST /calls renvoie 409 +BUSY si callee busy
- Frontend : filtre Tous / Manqués dans CallsScreen
- Ajouter durée sonnerie dans CallRecord (ringDuration)
- Analytics : taux décroché, durée moyenne, rejet
- Sécurité : chiffrer SDP, valider callId côté serveur avant ICE
- UI : animation avatar waves (CallAvatarWaves déjà présent)

14. Annexes : extraits de code

Modèle CallRecord

```
class CallRecord {
  final String id, convId;
  final CallType type; // AUDIO VIDEO
  final CallStatus status; // RINGING ONGOING ENDED MISSED REJECTED
  final bool isOutgoing, isGroup;
  final String? peerName, peerNumber, peerAvatarUrl;
  final int? participantCount;
  final DateTime startedAt;
  final DateTime? answeredAt, endedAt;
  final int? durationSec;
}

class IncomingCallInfo {
  final String callId, convId;
  final CallType callType;
  final String callerId, callerName;
  final String? callerAvatarUrl;
  final bool isGroup;
  final String? groupName;
  String get displayTitle => isGroup ? (groupName ?? "Appel de groupe") : callerName;
}
```

Routes backend

```
// GET /api/calls - historique 50 derniers
const parts = await prisma.callParticipant.findMany({
  where:{userId}, orderBy:{call:{startedAt:'desc'}}, take:50,
  include:{call:{include:{initiator:true, participants:{include:{user:true}}}}}
});

// POST /api/calls - démarre
const busy = await prisma.callParticipant.findFirst({
  where:{userId, joinedAt:{not:null}, leftAt:null, call:{status:{in:['RINGING','ONGOING']}}}
});
if(busy) return fail('Vous êtes déjà en appel',409,'BUSY');
```

Fix appel nom A affiche Appel chez B

```
// call_controller.dart
void acceptIncoming() {
  final inc = incoming;
  if(inc==null) return;
  activePeerName = inc.callerName; // pas displayTitle vide
  participantNames[inc.callerId] = inc.callerName;
  joinedParticipantIds.add(inc.callerId);
  incoming = null;
  activeCallId = inc.callId;
}

// active_call_screen.dart
String getName(CallController cc){
  if(widget.incoming && cc.incoming!=null) return cc.incoming!.displayTitle;
  if(primaryId!=null) return cc.participantNames[primaryId] ?? cc.activePeerName ??
  "Contact";
  return cc.activePeerName ?? "Contact";
}
```

Fin du cours – Version générée automatiquement depuis la branche arena/019fbc55-alanya le 01/08/2026. L'ensemble du mécanisme a été testé en bricolage frontend, backend restant en lecture seule (backup/pre-harmonisation-2026-07-24). Build APK release : flutter build apk --release (fix AppBar subtitle déjà appliqué).